PROYECTO ESPECIALIZACIÓN EN HERRAMIENTAS DE TESTING

SPRINT 2: Diseño de Pruebas y Primeras Automatizaciones

Sistema: Plataforma de Cadetería (Backend REST API)

Grupo 5

---

1. Análisis del Sistema Bajo Prueba

El sistema evaluado consiste en una plataforma integral de cadetería compuesta por una API REST/WebSocket desarrollada en Java 21 y Spring Boot 3.3.5, acoplada a una aplicación móvil nativa Android para cadetes.

Puntos Críticos y Arquitectura de Control:

1. Manejo de Sesión Única y Tokens JWT: El backend impone una restricción de sesión unívoca por cadete. Al realizar una autenticación exitosa (POST/api/auth/login/cadete), cualquier token de acceso generado previamente es invalidado de forma automática por la API, retornando 401 NO_AUTENTICADO ante solicitudes con tokens anteriores.

2. Control de Acceso Basado en Roles (RBAC): Los endpoints del panel de administración (/api/admin/*) requieren estrictamente el rol ADMIN. Si un usuario autenticado con rol CADETE intenta consumir estas rutas, la API detiene la solicitud respondiendo con 403 SIN_PERMISO. De manera simétrica, las rutas exclusivas de cadetes (/api/pedidos/me/*) rechazan solicitudes provenientes de tokens de administradores.

4. Análisis de Resultados y Lecciones Aprendidas

- **¿Qué salió bien?** La suite logró una tasa de éxito del 100% en los 15 casos ejecutados de manera automatizada. Se implementó una lógica de encadenamiento dinámico mediante las solicitudes de preparación (P1, P2, P3), garantizando que la suite obtenga y actualice automáticamente los tokens JWT y los identificadores de registros sin necesidad de intervención manual.

- **¿Qué fue más complicado de lo esperado?** El control de la caducidad del token por sesión concurrente. Fue necesario secuenciar estratégicamente la obtención del token del caso 06 (tokenVencido) antes de ejecutar el login del caso 01, asegurando así la simulación correcta de un token invalidado por inicio de sesión posterior.

- **¿Qué descubrieron sobre Postman/Testing?** La versatilidad de los scripts en la pestaña Tests para evaluar respuestas con código de estado HTTP 4xx y realizar aserciones detalladas sobre la estructura JSON de los errores de negocio (fieldErrors, codigo y message).

- **¿Qué cambiarían si tuvieran que empezar de nuevo?** Incorporar desde el inicio la automatización de reportes mediante la CLI de Newman integrada en un pipeline básico de CI/CD para ejecutar las pruebas tras cada despliegue del backend.

3. Matriz de Trazabilidad

Req. ID	Endpoint	Caso ID	Técnica Aplicada	HTTP	Código Error Esperado
REQ-AUTH-01	POST /api/auth/login/cadete	CP-01	Flujo Positivo	200	—
REQ-AUTH-02	POST /api/auth/login/cadete	CP-02	Credenciales Inválidas	401	CREDENCIALES_INVALIDAS
REQ-VAL-01	POST /api/auth/login/cadete	CP-03	Formato Tipo Dato	400	VALIDACION
REQ-CAD-01	GET /api/cadetes/me	CP-04	Consulta Perfil	200	—
REQ-SEC-01	GET /api/cadetes/me	CP-05	Token Mal Formado	401	NO_AUTENTICADO
REQ-SEC-02	GET /api/cadetes/me	CP-06	Token Vencido/Previo	401	NO_AUTENTICADO
REQ-ADM-01	GET /api/admin/cadetes/{id}	CP-07	Búsqueda por ID	200	—
REQ-ADM-02	GET /api/admin/cadetes/{id}	CP-08	Recurso Inexistente	404	NO_ENCONTRADO
REQ-SEC-03	GET /api/admin/cadetes/{id}	CP-09	Rol Insuficiente (Cadete)	403	SIN_PERMISO
REQ-CAD-02	PATCH /api/cadetes/me/estado	CP-10	Cambio de Estado Válido	200	—
REQ-VAL-02	PATCH /api/cadetes/me/estado	CP-11	Valor Enum Inválido	400	VALIDACION
REQ-SEC-04	PATCH /api/cadetes/me/estado	CP-12	Solicitud Sin Token	401	NO_AUTENTICADO
REQ-PED-01	GET /api/pedidos/me/{id}	CP-13	Detalle Pedido Propio	200	—
REQ-PED-02	GET /api/pedidos/me/{id}	CP-14	Pedido Inexistente	404	NO_ENCONTRADO
REQ-SEC-05	GET /api/pedidos/me/{id}	CP-15	Rol Incorrecto (Admin)	403	SIN_PERMISO

3. Validación de Enumerados y Esquemas de Datos:

Endpoints de actualización de estado (PATCH /api/cadetes/me/estado) y login requieren payloads estructurados. Ante datos fuera de los dominios permitidos (ej. estados diferentes a LIBRE, OCUPADO o DESCONECTADO, o letras en campos numéricos como versionApp), la API responde con un código de estado 400 VALIDACION identificando explícitamente el campo defectuoso en fieldErrors.

2. Estrategia de Prueba y Técnicas Aplicadas

Se diseñaron 15 casos de prueba ejecutables sobre 5 endpoints clave (3 casos por endpoint: 1 flujo positivo y 2 de error), cubriendo códigos de respuesta HTTP de éxito (200 OK) y de error (400, 401, 403, 404).

Técnicas de Diseño Utilizadas:

- Pruebas de Transición de Estados: Verificación de cambios de disponibilidad del cadete y consulta de historial de pedidos.

- Análisis de Valores Límite y Dominio: En envío de tipos de datos incorrectos a nivel de contrato DTO (versionApp, estadoId).

- Pruebas de Seguridad e Inyección de Rol: Verificación de denegación de acceso cuando se combinan tokens de cadete en rutas admin y viceversa.